# A CAPSTONE PROJECT REPORT

# AGRIADVISOR

## A Multi-Agent, Rule-Based Expert System for Crop Disease Diagnosis and Farm Resource Advisory

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

**MLA0101-ARTIFICIAL INTELLIGENCE FOR EXPERT SYSTEMS**

to the award of the degree of

**BACHELOR OF TECHNOLOGY**

IN

**COMPUTER SCIENCE ENGINEERING**

**(CSE)**


**Submitted by**

Shaik Mubarak(192425194)

Sydhusagari Shaik Aman Hussain(192425021)

Y Neeraj(192425043)

Kampati Pavan Kumar (192524333)

**Under the Supervision of**

**Dr. P. Subramanian**

**SIMATS ENGINEERING**

**Saveetha Institute of Medical and Technical Sciences**

Chennai-602105


SEPTEMBER 2026

**SIMATS ENGINEERING**
**Saveetha Institute of Medical and Technical Sciences**
**Chennai-602105**

# BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled **"AgriAdvisor"** has been carried out by **Shaik Mubarak(192425194)** under the supervision of **Dr. P. Subramanian** and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Artificial Intelligence and Machine learning programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

| COURSE COORDINATOR | COURSE FACULTY |
|---|---|
| Dr. M. Prakash<br>Dept. of CSE (Quantom Intelligence)<br>SIMATS Engineering, Chennai – 602105 | Dr. P. Subramanian<br>Professor<br>Dept. of CSE (Information Security)<br>SIMATS Engineering, Chennai – 602105 |

**SIMATS ENGINEERING**
**Saveetha Institute of Medical and Technical Sciences**
**Chennai-602105**

# DECLARATION

I **Shaik Mubarak(192425194)** of the B.Tech Artificial Intelligence and Machine learning programme Department, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled **"A Multi-Agent, Rule-Based Expert System for Crop Disease Diagnosis and Farm Resource Advisory"** is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the test results and analysis presented in this report correspond to the implemented software project and are not intentionally fabricated, falsified, or manipulated. Any external tools, software, computational resources, or AI-assisted tools used during development have been appropriately disclosed. We confirm that applicable academic and institutional requirements have been followed throughout the planning, implementation, testing, and documentation of the project.

**Place:** Chennai
**Date:** 04/09/2026

**Signature of the Student:**Shaik Mubarak(192425194)

**Individual Contribution Statement**

| Student Name / Register No. | Specific Responsibilities | Design & Development Contribution | Testing & Analysis Contribution | Report Contribution | Approx. Contribution (%) |
|---|---|---|---|---|---|
| **Shaik Mubarak(192425194)** | **Module 1: Irrigation Advisory Agent; PEAS and utility-based decision design.** | **Designed irrigation percepts, utility scoring, irrigation rules and CLI inputs.** | **Verified low/adequate moisture and rainfall-delay scenarios.** | **Ch. 1 background/objectives; irrigation module documentation; Appendix D.** | **25%** |
| Y Neeraj(192425043) | Module 2: Expert System Rule Engine; IF–THEN knowledge base. | Developed the 40-rule knowledge base, forward chaining and conflict resolution. | Verified rule firing and reasoning traces across test cases. | Ch. 2–3 literature/design sections; Appendix C rule logic. | 25% |
| Sydhusagari Shaik Aman Hussain(192425021) | Module 3: Pest Alert Agent; model-based risk advisory. | Designed internal world model, pest-risk rules and alert outputs. | Tested temperature/humidity/pest-count risk scenarios. | Ch. 4 results/testing; pest-agent evidence. | 25% |
| Kampati Pavan Kumar (192524333) | Module 4: Disease Diagnosis Agent; integration and testing. | Designed goal-based diagnosis, integrated all agents and final farm report. | Executed end-to-end scenarios and analysed outputs. | Ch. 5; project planning; appendices and final compilation. | 25% |
| TOTAL | | | | | 100% |

# ABSTRACT

Small-scale farmers often depend on field-agent advice that may be delayed, inconsistent, or unavailable when irrigation, pest pressure, or crop symptoms change quickly. AgriAdvisor is an AI-based farm decision-support prototype that addresses this problem using three cooperating software agents: an Irrigation Advisory Agent, a Pest Alert Agent, and a Crop Disease Diagnosis Agent. The agents collect or receive structured farm percepts and query a central rule-based expert system containing 40 IF-THEN production rules.

The Irrigation Advisory Agent uses a utility-based strategy to balance soil-moisture deficit, crop growth stage, temperature stress, and rainfall expectations. The Pest Alert Agent uses a model-based reflex strategy and maintains an internal world model containing previous pest risk, pest counts, and environmental trends. The Disease Diagnosis Agent uses a goal-based strategy to identify the most likely disease from crop symptoms and environmental evidence. The central inference engine uses forward chaining, rule specificity, numeric priority, and duplicate/cycle prevention to derive conclusions and recommendations.

The implemented system is a local Python application with a command-line interface and automated tests. Ten defined test cases cover multiple crops, diseases, irrigation conditions, pest-risk conditions, and healthy states. The reported automated suite completed 10/10 tests successfully, giving 100.0% pass accuracy for the implemented test set. The project demonstrates the complete chain from percepts and agent reasoning to expert-system inference and farmer-oriented recommendations.



**Keywords**: Artificial Intelligence, Multi-Agent System, Expert System, IF-THEN Rules, Forward Chaining, Crop Disease Diagnosis, Irrigation Advisory, Pest Alert, AgriAdvisor

## TABLE OF CONTENTS



## LIST OF FIGURES



## LIST OF TABLES



## LIST OF ABBREVIATIONS / SYMBOLS

| Symbol | Description | Unit |
|---|---|---|
| AI | Artificial Intelligence | — |
| PEAS | Performance, Environment, Actuators, Sensors | — |
| KB | Knowledge Base | — |
| IF–THEN | Conditional production rule representation | — |
| CLI | Command-Line Interface | — |

| Symbol | Description | Unit |
|---|---|---|
| U | Utility score | Normalized |
| TC | Test Case | — |

# CHAPTER 1

# INTRODUCTION AND ENGINEERING PROBLEM

## 1.1 Background

Agricultural decisions are time-sensitive. Soil moisture can fall below a useful level within a short period, pest populations can increase under favourable environmental conditions, and visible symptoms may require early diagnosis. Small-scale farmers may not always have immediate access to consistent field-agent advice. An intelligent decision-support system can reduce this delay by continuously accepting farm observations and applying explicit reasoning rules.

AgriAdvisor uses a multi-agent architecture in which specialised software agents focus on irrigation, pest alerts, and crop-disease diagnosis. A central rule-based expert system provides a transparent reasoning layer. This separation allows each agent to have its own percepts, actions, and internal state while sharing a common knowledge base and inference engine.

## 1.2 Need for the Project

- Manual farm advice can be delayed, especially when environmental conditions change quickly.
- Different farm decisions require different reasoning strategies rather than a single generic reaction.
- Rule-based reasoning is explainable because recommendations can be linked to explicit IF-THEN rules.
- A central expert system avoids duplicated decision logic across agents.
- A low-cost local software prototype provides a practical platform for demonstrating AI agent concepts and expert-system implementation.

## 1.3 Problem Statement

Design and implement AgriAdvisor, an AI-based farm decision-support system for small-scale farmers that supports irrigation scheduling, pest/disease alerts, and crop-disease diagnosis. The system must formulate each sub-task using PEAS, justify an appropriate agent strategy based on observability, determinism, and environmental dynamics, implement software agents with defined percepts, actions and internal state, and integrate them with a rule-based expert system using IF-THEN production rules and forward chaining.

## 1.4 Project Aim

To develop a modular multi-agent farm advisory prototype that transforms structured crop and environmental observations into explainable irrigation, pest-risk, and disease-diagnosis recommendations using a rule-based expert system.

## 1.5 Project Objectives

1. To formulate irrigation, pest-alert, and disease-diagnosis tasks using PEAS.
2. To select and justify suitable agent strategies for each sub-task.
3. To design software agents with clear percepts, actions, internal states, and goals.
4. To construct a structured knowledge base containing at least 40 IF-THEN production rules.
5. To implement a forward-chaining inference engine with conflict resolution and reasoning traces.
6. To integrate all three agents with the expert system for end-to-end farm advisory.
7. To test the implementation across diverse crop, disease, irrigation, pest-risk, and healthy scenarios.

## 1.6 Scope

Included:

- Python-based local decision-support application.
- Three software agents for irrigation, pest alerts, and disease diagnosis.

- Central rule-based knowledge base with 40 production rules.
- Forward-chaining inference with reasoning trace.
- CLI-based individual-agent and complete-farm analysis.
- Automated functional test cases.

Excluded:

- Certified agricultural diagnosis or professional agronomist replacement.
- Autonomous pesticide application or chemical dosage control.
- Guaranteed disease diagnosis from images without a validated vision model.
- Real-time field deployment without calibrated sensors and agronomic validation.
- Guaranteed crop yield or water-saving performance beyond the tested software scenarios.

### 1.7 Expected Engineering Outcome

The expected outcome is a working software prototype in which farm percepts are processed by specialised agents, passed to the rule-based expert system, evaluated through forward chaining, and converted into transparent recommendations. The system should also expose the rules fired so that the reasoning process can be inspected during demonstration and viva.

**Table 1.1 Task, Requirement, Specific Parameters, Outcome and Techniques**

| Task | Requirement | Specific Parameters | Outcome | Techniques |
|---|---|---|---|---|
| Irrigation Advisory Agent | Recommend when to irrigate or delay irrigation | Soil moisture, rainfall, temperature, humidity, crop, growth stage | Irrigate / delay / monitor | Utility-based agent + rules |
| Pest Alert Agent | Estimate pest risk and recommend monitoring/control | Temperature, humidity, rainfall, pest count, crop stage, damage | Low / medium / high risk | Model-based reflex + world model |
| Disease Diagnosis Agent | Identify likely crop disease and action | Crop, leaf colour, spots, wilting, lesions, curling, environment | Disease + recommendation | Goal-based agent + rules |
| Expert System | Reason over all agent facts | 40 IF-THEN production rules | Derived facts and recommendations | Forward chaining + conflict resolution |

# CHAPTER 2

# LITERATURE REVIEW AND EXISTING SOLUTIONS

## 2.1 Review Method

The review is organised around farm decision-support, intelligent agents, rule-based expert systems, and agricultural advisory applications. The proposed design emphasises explainability, modularity, and suitability for small-scale decision support rather than attempting to replace specialised agronomic services.

## 2.2 Review of Existing Work

Traditional farm advisory systems often rely on fixed schedules, manual observations, or direct expert consultation. Sensor-assisted agriculture improves access to environmental observations, while machine-learning approaches can support prediction when sufficient labelled data are available. Rule-based expert systems remain useful where decisions can be expressed through domain rules and where transparent explanations are important.

Agent-based approaches further separate responsibilities. An irrigation agent can focus on water decisions, a pest agent can maintain environmental and historical context, and a diagnosis agent can pursue the goal of identifying the most likely disease. AgriAdvisor combines these ideas with a shared inference engine so that specialised agents can reuse common facts and recommendations.

## 2.3 Comparative Analysis

| Existing Approach | Advantages | Limitations | Relevance to Proposed Work |
|---|---|---|---|
| Manual field-agent advice | Human expertise and contextual judgement | May be delayed or inconsistent; difficult to scale | Problem addressed by automated local reasoning |
| Fixed irrigation schedule | Simple to implement | Ignores changing soil/weather conditions | Baseline for irrigation comparison |
| Sensor-only advisory | Provides current observations | Does not itself explain complex decisions | Sensors become percepts for agents |
| Machine-learning advisory | Can model complex patterns | Needs data, training and validation | Possible future extension |
| Rule-based expert system | Explainable, deterministic, easy to inspect | Depends on quality/completeness of rules | Core reasoning layer |
| Multi-agent advisory | Modular responsibilities and specialised strategies | Requires integration and coordination | Selected architecture for AgriAdvisor |

## 2.4 Research / Engineering Gap

A simple farm sensor dashboard can show measurements, but it does not necessarily convert those measurements into a coherent, explainable recommendation. Conversely, a single monolithic rule engine may not demonstrate how different AI agent strategies suit different sub-problems. AgriAdvisor addresses this gap by explicitly separating agent strategy from the shared expert-system reasoning layer.

## 2.5 Proposed Contribution

The proposed contribution is a modular Python prototype that demonstrates PEAS-based problem formulation, strategy selection, three software-agent architectures, a 40-rule knowledge base, forward-chaining inference, conflict resolution, and end-to-end advisory. The architecture is designed for academic demonstration and can later be extended with calibrated IoT sensors, image-based diagnosis, weather services, and learning components.

# CHAPTER 3

# AGENT STRATEGY, ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

## 3.1 PEAS Analysis

| Agent | Performance Measure | Environment | Actuators | Sensors / Percepts |
|---|---|---|---|---|
| Irrigation Advisory Agent | Adequate soil moisture, reduced unnecessary irrigation, crop support, water conservation | Farm field; partially observable, stochastic, dynamic, sequential | Irrigate, stop/delay irrigation, send recommendation | Soil moisture, temperature, humidity, rainfall, crop type, growth stage |
| Pest Alert Agent | Early pest-risk detection, reduced crop damage, reduced unnecessary control actions | Crop field; partially observable, stochastic, dynamic | Risk alert, monitoring recommendation, pest-control advisory | Temperature, humidity, rainfall, crop type/stage, leaf damage, pest count |
| Disease Diagnosis Agent | Accurate likely-disease identification and appropriate recommendation | Plant/crop; partially observable, stochastic, dynamic | Diagnosis, treatment/action recommendation, warning, monitoring | Crop, leaf colour, spots, wilting, lesions, curling, temperature, humidity |

## 3.2 Environment and Agent Strategy Justification

| Sub-task | Observability | Determinism / Dynamics | Selected Strategy | Justification |
|---|---|---|---|---|
| Irrigation | Partially observable | Stochastic, dynamic, sequential | Utility-Based | Balances competing objectives such as moisture deficit, crop stage, temperature stress and rainfall expectation rather than reacting to one percept. |
| Pest Alert | Partially observable | Stochastic, dynamic | Model-Based Reflex | Maintains an internal world model using previous risk/count information and combines it with current percepts. |
| Disease Diagnosis | Partially observable | Stochastic, dynamic | Goal-Based | Pursues the explicit goal of identifying the most likely disease and deriving an action plan from symptoms and context. |

### 3.3 Irrigation Advisory Agent Design

Percepts: soil moisture, temperature, humidity, rainfall, crop type, growth stage, and rainfall expectation. Internal state stores the latest percepts, previous soil-moisture value, irrigation status, crop water requirement, and utility score. Actions include START_IRRIGATION, STOP_IRRIGATION, DELAY_IRRIGATION, and SEND_IRRIGATION_ALERT.

The implemented utility score is normalised to $U \in [-1.0, 1.0]$. The score combines moisture deficit, temperature stress, growth-stage importance, and rainfall discount. The score is then interpreted together with expert-system rules so that the agent's numerical preference and explicit domain knowledge both contribute to the final advisory.

### 3.4 Pest Alert Agent Design

Percepts include temperature, humidity, rainfall, crop type, crop stage, leaf damage, and pest count. Internal state maintains previous pest risk, previous pest count, current environmental conditions, and a population trend such as increasing_rapidly, stable, or decreasing. Actions include LOW_RISK, MEDIUM_RISK, HIGH_RISK, SEND_PEST_ALERT, and RECOMMEND_CONTROL.

### 3.5 Disease Diagnosis Agent Design

Percepts include crop type and visible symptoms such as leaf colour, brown spots, wilting, lesions, and leaf curling, along with temperature and humidity. Internal state stores observed symptoms, candidate diseases, and diagnostic evidence. The goal is to identify the most likely disease and recommend appropriate non-prescriptive management actions.

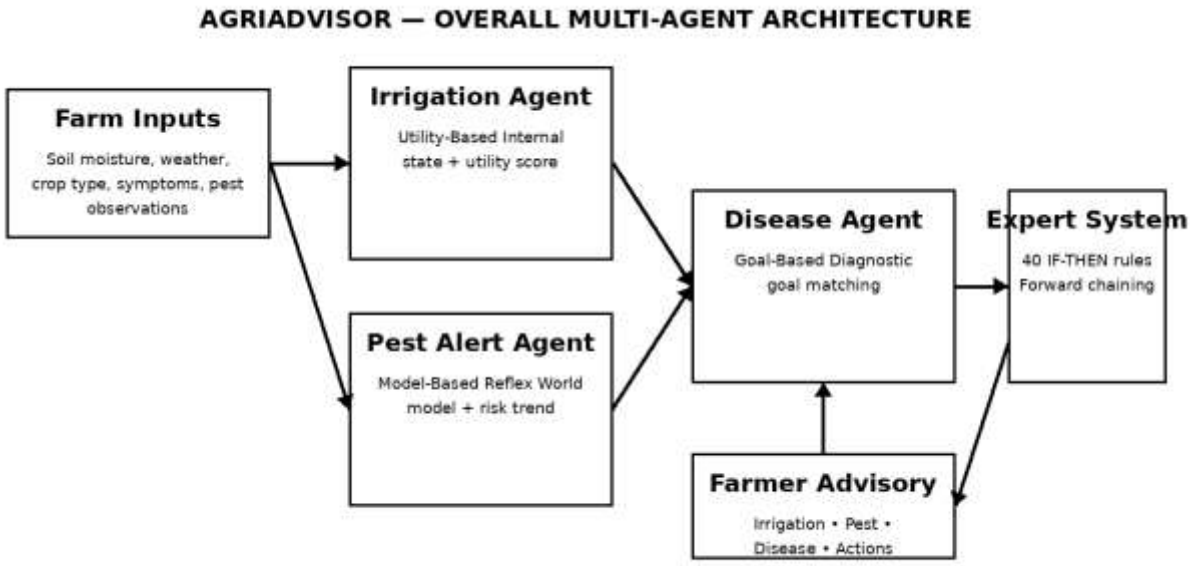



**Figure 3.1 – Overall architecture of the AgriAdvisor multi-agent system**

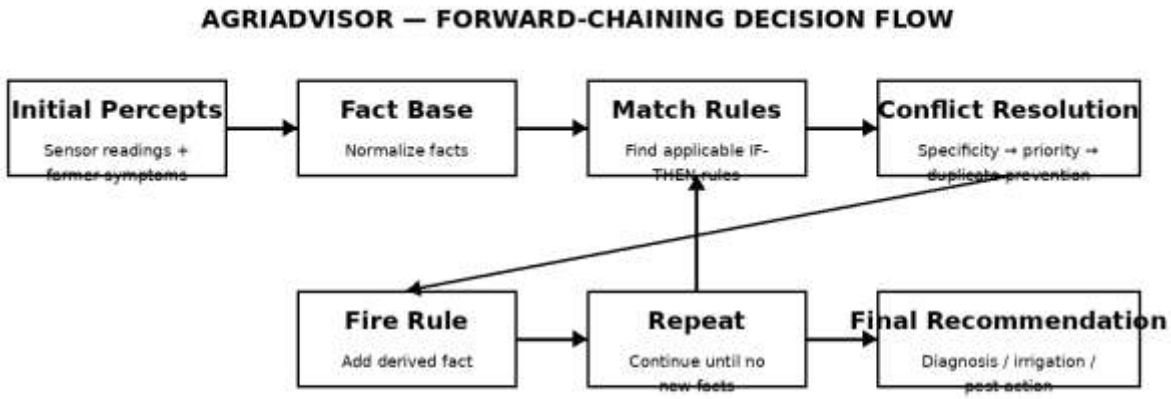



**Figure 3.2 – Forward-chaining expert-system decision flow**

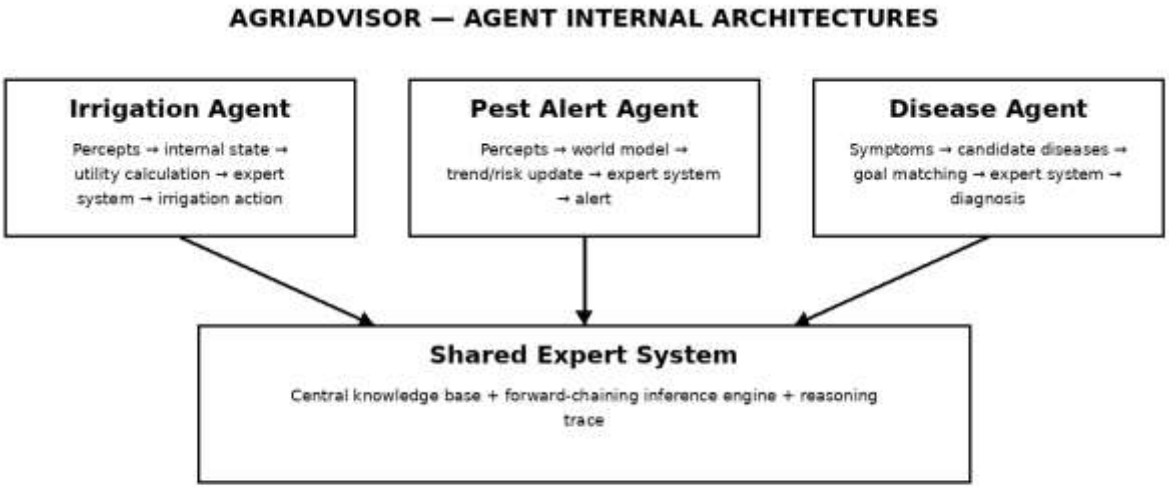



**Figure 3.3 – Internal architectures of the three software agents**

### 3.6 Expert-System Architecture

The expert system consists of a knowledge base and a forward-chaining inference engine. Initial percepts are converted into facts. The inference engine searches for rules whose conditions are satisfied, fires applicable rules, adds newly derived facts, and continues until no additional facts can be derived. A reasoning trace records FACT, FIRED, and DERIVED events for explainability.

### 3.7 Knowledge Representation

Example production rule:
IF crop = tomato
AND leaf_spots = brown
AND humidity = high
THEN disease = early_blight

Example recommendation rule:
IF disease = early_blight
THEN action = remove_affected_leaves

### 3.8 Conflict Resolution

8. Rule specificity: rules matching more conditions are preferred.
9. Numeric priority: when specificity is equal, the rule priority value is considered.
10. Duplicate/cycle prevention: derived facts already present in the fact set are not repeatedly added.

### 3.9 Requirements

| Requirement | Type | Measurable Target |
|---|---|---|
| Process irrigation percepts | Functional | Correct irrigation action for defined test scenarios |
| Process pest percepts | Functional | Correct risk classification for defined scenarios |
| Process disease symptoms | Functional | Correct expected diagnosis for defined scenarios |
| Apply expert rules | Functional | Applicable IF-THEN rules fire correctly |
| Provide reasoning trace | Functional | FACT/FIRED/DERIVED events visible |
| Stable operation | Non-Functional | No unhandled exception during planned tests |
| Explainability | Non-Functional | Recommendation can be linked to fired rules |
| Low-cost prototype | Non-Functional | Runs with standard Python 3 libraries |

### 3.10 Design Specifications

| Parameter | Target / Requirement | Priority |
|---|---|---|
| Agent layer | Three specialised software agents | High |
| Knowledge base | 40 IF-THEN rules | High |
| Inference engine | Forward chaining with conflict resolution | High |
| Irrigation strategy | Utility-Based | High |
| Pest strategy | Model-Based Reflex | High |
| Disease strategy | Goal-Based | High |
| Interface | CLI individual-agent and complete-farm modes | Medium |
| Testing | 10 automated test cases | High |

### 3.11 Alternative Solutions & Evaluation

| Criterion | Rule-Based Multi-Agent | Single Monolithic Rule Engine | ML-Only Advisory |
|---|---|---|---|
| Explainability | High | High | Medium/Low |
| Data requirement | Low | Low | High |
| Modularity | High | Medium | Medium |

| Criterion | Rule-Based Multi-Agent | Single Monolithic Rule Engine | ML-Only Advisory |
|---|---|---|---|
| Ease of viva demonstration | High | High | Medium |
| Adaptability without retraining | High | High | Low |
| Selected | Yes | No | Future extension |

### 3.12 Trade-off Analysis

| Design Decision | Alternative Considered | Benefit | Disadvantage | Final Decision |
|---|---|---|---|---|
| Multi-agent architecture | Single agent | Clear specialisation and strategy mapping | More integration work | Selected to satisfy CO3/CO4 |
| Rule-based reasoning | ML-only model | Transparent and deterministic | Rule coverage depends on KB | Selected as core CO5 component |
| Local Python CLI | Cloud application | Simple, low-cost and offline | Limited remote access | Selected for academic prototype |
| Forward chaining | Backward chaining only | Natural for deriving multiple recommendations from sensor facts | May fire more rules than necessary | Selected for end-to-end fact-driven advisory |

### 3.13 Sustainability, Ethics, Safety, Risk & Security

| Domain | Consideration | Evidence / Design Response | Impact |
|---|---|---|---|
| Sustainability | Avoid unnecessary resource use | Water-saving irrigation logic and modular software | Supports responsible resource use |
| Ethics | Avoid overstating AI certainty | Use 'likely disease' wording and recommend expert confirmation for unusual cases | Reduces misleading advice |
| Safety | Avoid hazardous autonomous actions | No autonomous chemical dosing or physical pesticide application | Keeps system advisory |
| Security | Protect rule configuration | Keep rule files controlled and documented | Reduces accidental rule changes |
| Privacy | Minimise farm data exposure | Local processing by default | No cloud transfer required |

**Table 3.8 Risk Register**

| Risk | Likelihood | Severity | Mitigation | Residual Risk |
|---|---|---|---|---|
| Incorrect disease rule match | Medium | High | Expand and validate symptom rules; use expert confirmation | Medium |

| Risk | Likelihood | Severity | Mitigation | Residual Risk |
|---|---|---|---|---|
| False pest alert | Medium | Medium | Use internal trend state and multiple conditions | Low–Medium |
| Incorrect irrigation recommendation | Medium | High | Combine utility score with explicit rules and rainfall condition | Low–Medium |
| Missing/ambiguous percept | Medium | Medium | Input validation and safe fallback recommendation | Low |
| User interprets advisory as guaranteed diagnosis | Medium | High | Clearly state prototype/advisory limitations | Low |
| Rule conflict | Low–Medium | Medium | Specificity, priority and duplicate prevention | Low |

# CHAPTER 4

# IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

## 4.1 Development Approach & Resources

The software was developed incrementally. The knowledge representation and inference engine were established first, followed by the three specialised agents. The command-line interface was then integrated with the agents and the expert system. Automated test cases were executed after integration so that individual and end-to-end behaviours could be checked.

**Table 4.1 Hardware / Software / Tools Used**

| Tool / Software / Resource | Purpose | Stage Used |
|---|---|---|
| Python 3 | Application development and execution | All stages |
| Standard Python libraries | Core implementation without external runtime dependencies | All stages |
| AgriAdvisor CLI | User interaction and demonstration | Testing / demonstration |
| Knowledge Base | 40 IF-THEN production rules | Inference |
| Forward-Chaining Inference Engine | Rule matching and fact derivation | Inference |
| Automated test suite | Functional verification | Testing |
| Git / GitHub repository | Version control and submission | Documentation / delivery |

## 4.2 Software Implementation

The implementation contains three agent classes and a central expert-system package. The agents receive structured input, update their internal state, query the inference engine, and return actions or recommendations. The knowledge base contains 40 IF-THEN production rules covering disease diagnosis, irrigation, pest risks, environmental conditions, and recommendations.

```
AgriAdvisor/
├── main.py
├── README.md
├── requirements.txt
├── agents/
│   ├── irrigation_agent.py
│   ├── pest_agent.py
│   └── disease_agent.py
├── expert_system/
│   ├── knowledge_base.py
│   └── inference_engine.py
├── tests/
│   └── test_cases.py
└── docs/
    ├── PEAS_Analysis.md
    ├── Agent_Design.md
    ├── Knowledge_Base.md
    ├── Architecture.md
    └── Testing_Results.md
```

### 4.3 Rule Knowledge Base

| Rule Category | Examples of Coverage | Approx. Rules |
|---|---|---|
| Disease diagnosis | Tomato Early Blight, Tomato Leaf Curl, Rice Blast, Rice Brown Spot, Chilli Leaf Curl, Powdery Mildew, Bacterial Leaf Spot, healthy status | 16 |
| Irrigation | Moisture deficit, adequate moisture, rainfall expectation, growth stage, high temperature | 10 |
| Pest alerts | Whitefly, thrips, caterpillar, aphids, humidity/temperature favourability, pest count | 10 |
| Recommendations | Monitoring, cultural/physical management, field actions | 4 |
| TOTAL | 40 IF-THEN production rules | 40 |

### 4.4 Test Plan & Verification

| Requirement | Test Method | Acceptance Criterion |
|---|---|---|
| Early-blight diagnosis | Tomato with brown spots, yellow leaves and high humidity | Expected diagnosis is Early Blight |
| Leaf-curl diagnosis | Tomato with leaf curling and pest indicators | Expected leaf-curl diagnosis / risk alert |
| Rice disease diagnosis | Rice with disease-specific symptoms | Expected corresponding rice disease |
| Chilli disease diagnosis | Chilli with leaf curling and pest indicators | Expected Chilli Leaf Curl / alert |
| Irrigation required | Low soil moisture with no rain | Irrigation recommendation |
| No irrigation | Adequate soil moisture | No immediate irrigation |
| Delay irrigation | Low moisture with rainfall expected | Delay irrigation |
| High pest risk | High temperature + humidity + high pest count | High pest risk |
| Healthy crop | Healthy conditions | Continue monitoring |
| End-to-end farm analysis | Combined crop/environment/symptom input | Combined advisory and rules fired |

### 4.5 Automated Test Results

The implemented automated suite contains ten executable test cases. The reported execution completed all ten cases successfully.

| Test Case | Scenario | Expected Result | Status |
|---|---|---|---|
| TC01 | Tomato Early Blight Diagnosis | Early Blight | PASS |
| TC02 | Tomato Leaf Curl Diagnosis | Leaf Curl | PASS |
| TC03 | Rice Blast Disease Diagnosis | Rice Blast | PASS |
| TC04 | Rice Brown Spot Diagnosis | Rice Brown Spot | PASS |
| TC05 | Chilli Leaf Curl Diagnosis | Chilli Leaf Curl | PASS |
| TC06 | Low Moisture + No Rain Irrigation | Irrigation Required | PASS |
| TC07 | Adequate Moisture Irrigation | No Immediate Irrigation | PASS |
| TC08 | Low Moisture + Rain Expected | Delay Irrigation | PASS |

| Test Case | Scenario | Expected Result | Status |
|---|---|---|---|
| TC09 | High Temp + Humidity + High Pest Count | High Pest Risk | PASS |
| TC10 | Healthy Crop Conditions | Continue Monitoring | PASS |

| Metric | Result |
|---|---|
| Total Test Cases | 10 |
| Passed | 10 |
| Failed | 0 |
| Overall Pass Accuracy | 100.0% |

### 4.6 Demonstration Scenario

A representative complete-farm scenario uses Tomato as the crop with soil moisture 25%, temperature 29°C, humidity 85%, rainfall 0 mm, brown leaf spots, yellow leaves, and a high pest count. The system processes the information through the three agents and the central expert system. The expected demonstration output is an irrigation recommendation, high pest-risk warning, and an Early Blight diagnosis with actionable field-management recommendations.

Example reasoning trace:
FACT: crop = tomato
FACT: soil_moisture = low
FACT: leaf_spots = brown
FACT: humidity = high
FIRED: Tomato Early Blight Detection
DERIVED: disease = early_blight
FIRED: Early Blight Recommendation
DERIVED: action = remove_affected_leaves
FIRED: Low Moisture Irrigation Rule
DERIVED: irrigation = required

### 4.7 Results & Engineering Analysis

The test results demonstrate that the rule engine can map defined percept combinations to the expected recommendations. The reasoning trace provides evidence that the result is not simply a hard-coded final label: facts lead to applicable rules, rules derive new facts, and derived facts can trigger additional recommendation rules. This supports the CO5 requirement for a functioning rule-based expert system.

The 100.0% figure is the pass rate of the implemented ten-case software test suite, not a claim of real-world agricultural diagnostic accuracy. Real-world deployment would require agronomic validation, larger datasets, sensor calibration, seasonal testing, and evaluation by qualified agricultural experts.

### 4.8 Comparison with Existing Solutions & Achievement of Objectives

| Objective | Evidence | Achievement |
|---|---|---|
| PEAS formulation | PEAS matrix for all three agents | Fully Achieved |
| Agent strategy selection | Utility-Based, Model-Based Reflex, Goal-Based justifications | Fully Achieved |
| Software agent design | Percepts, actions, internal states and architectures | Fully Achieved |
| IF-THEN expert system | 40-rule knowledge base | Fully Achieved |

| Objective | Evidence | Achievement |
|---|---|---|
| Inference engine | Forward chaining + conflict resolution + trace | Fully Achieved |
| Agent integration | Complete farm analysis | Fully Achieved |
| Testing | 10 automated test cases | Fully Achieved |

### 4.9 Strengths & Limitations

Strengths:

- Three-agent architecture directly demonstrates different agent strategies.
- IF-THEN rules are transparent and easy to explain during viva.
- Forward-chaining trace makes the reasoning process visible.
- Modular Python structure supports testing and maintenance.
- Local execution does not require external runtime services.

Limitations:

- Rules represent a prototype knowledge base and cannot cover every crop or disease.
- Diagnosis is based on structured symptoms rather than validated image analysis.
- Environmental inputs are user-entered in the CLI prototype rather than continuously measured field sensors.
- The 100% test pass rate is limited to the ten implemented cases and does not represent real-world accuracy.
- Recommendations should be validated by agricultural professionals before field deployment.

### 4.10 Project Planning, Roles & Budget

| Milestone | Planned Activity | Expected Evidence |
|---|---|---|
| 1 | Problem definition and PEAS formulation | Problem statement, PEAS and objectives |
| 2 | Agent strategy and architecture | Agent design and architecture diagrams |
| 3 | Knowledge-base construction | 40-rule catalog |
| 4 | Inference-engine implementation | Forward-chaining code and trace |
| 5 | Agent integration | CLI and complete farm analysis |
| 6 | Testing and validation | 10 test cases and results |
| 7 | Documentation and demonstration | Final report and screenshots |

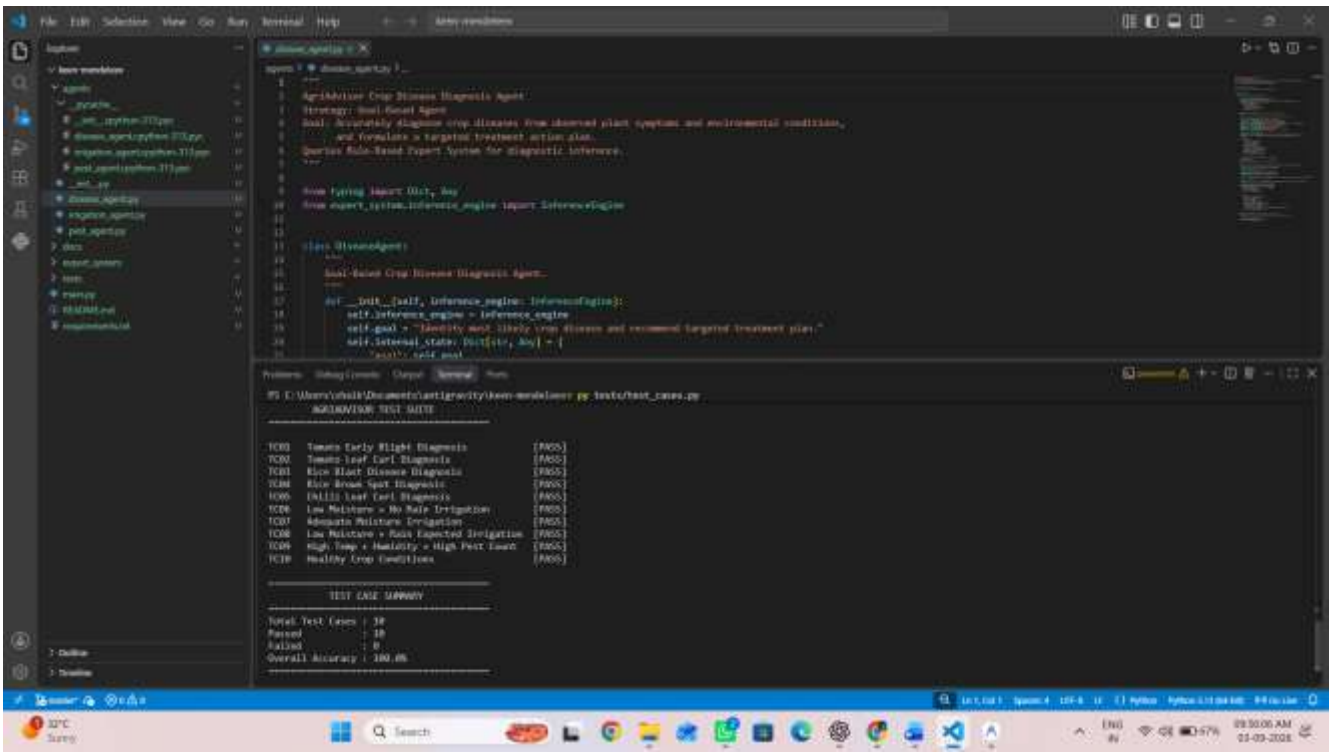
| Student | Assigned Responsibility | Actual Contribution |
|---|---|---|
| Shaik Mubarak(192425194) | Irrigation Advisory Agent | Irrigation strategy, rules, tests and documentation |
| Y Neeraj(192425043) | Expert System | Knowledge base, inference engine and rule verification |
| Sydhusagari Shaik Aman Hussain(192425021) | Pest Alert Agent | World model, pest rules, alerts and testing |
| Kampati Pavan Kumar (192524333) | Disease Agent & Integration | Goal-based diagnosis, integration, testing and final compilation |

| Item | Estimated Cost | Actual Cost |
|---|---|---|
| Python / standard libraries | ₹0 | ₹0 |
| Development computer | Existing | Existing |

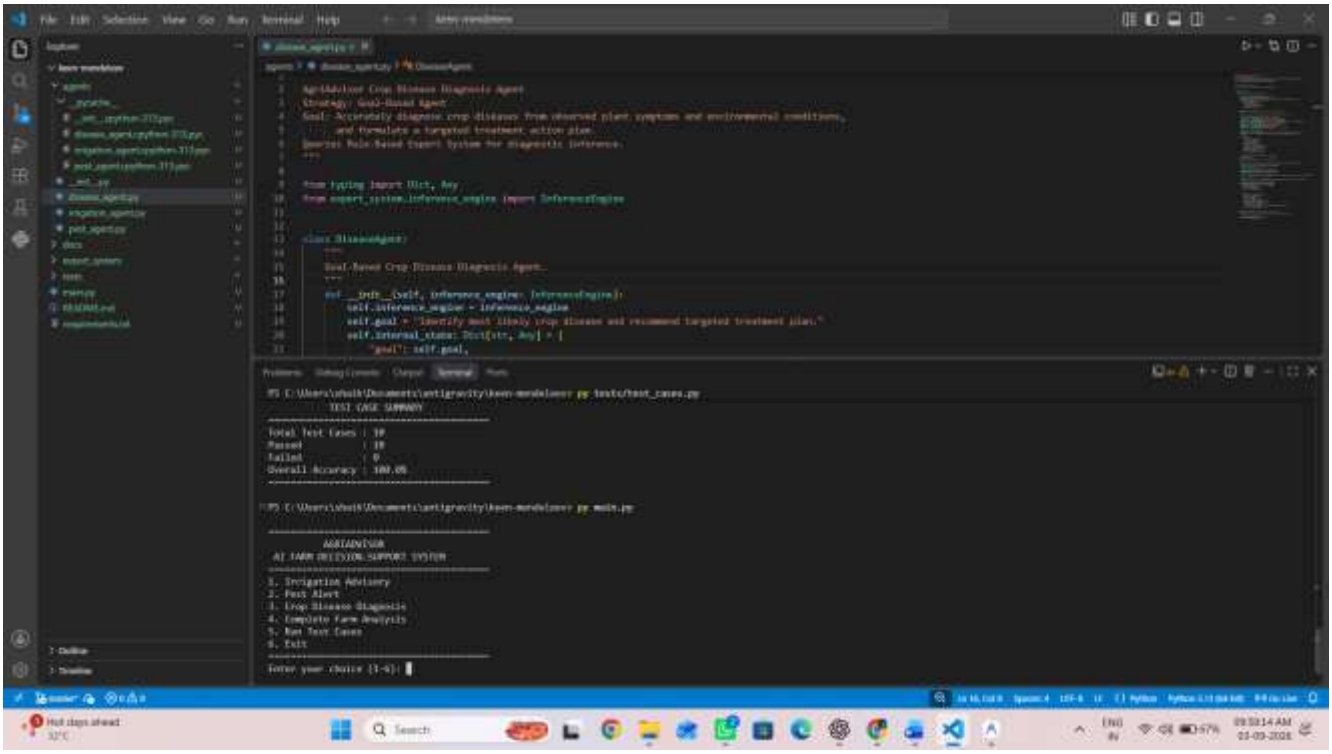
| Item | Estimated Cost | Actual Cost |
|---|---|---|
| Git / GitHub | ₹0 for academic repository | ₹0 |
| Optional future sensors | To be determined | Not part of base software prototype |
| TOTAL | To be finalised for any physical deployment | Not applicable to base software prototype |

### 4.11 Demonstration Evidence

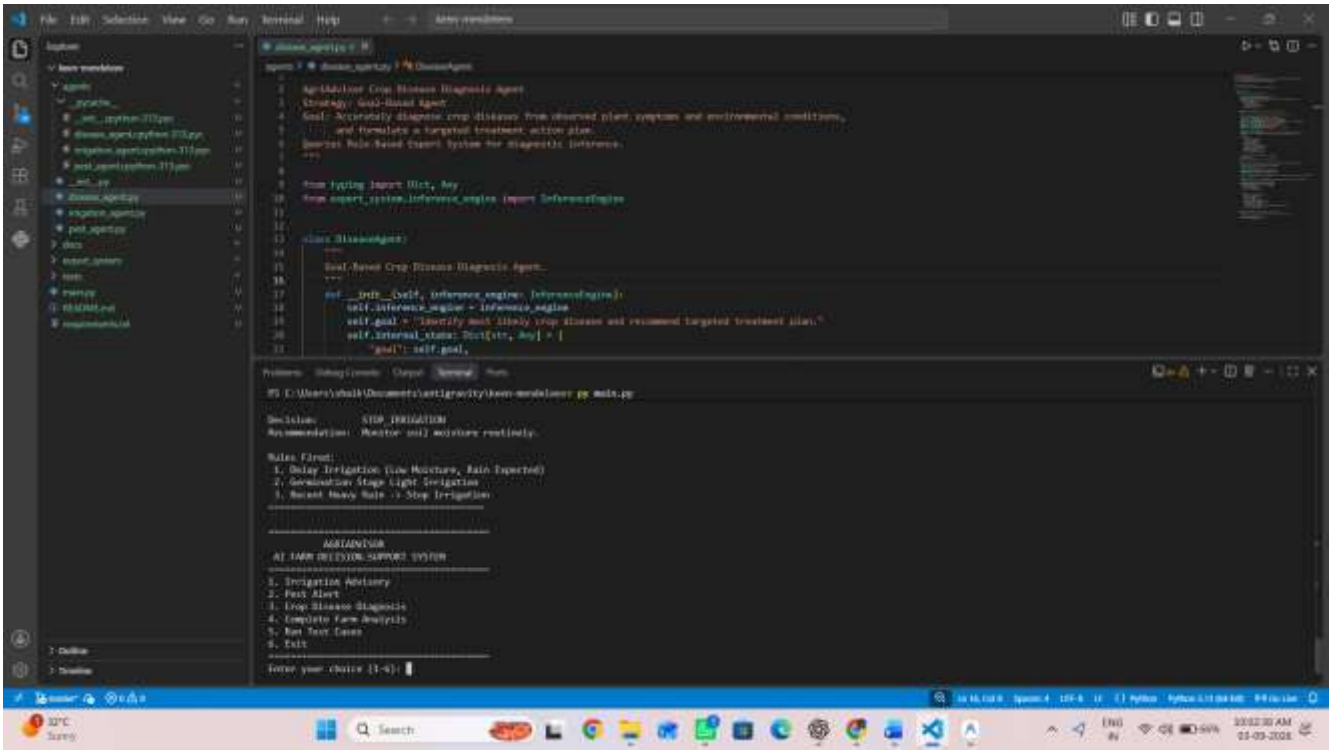
The following evidence should be inserted after running the actual application in Antigravity/terminal. The screenshots are deliberately left as evidence slots so that the submitted report contains the team's real outputs rather than invented images.



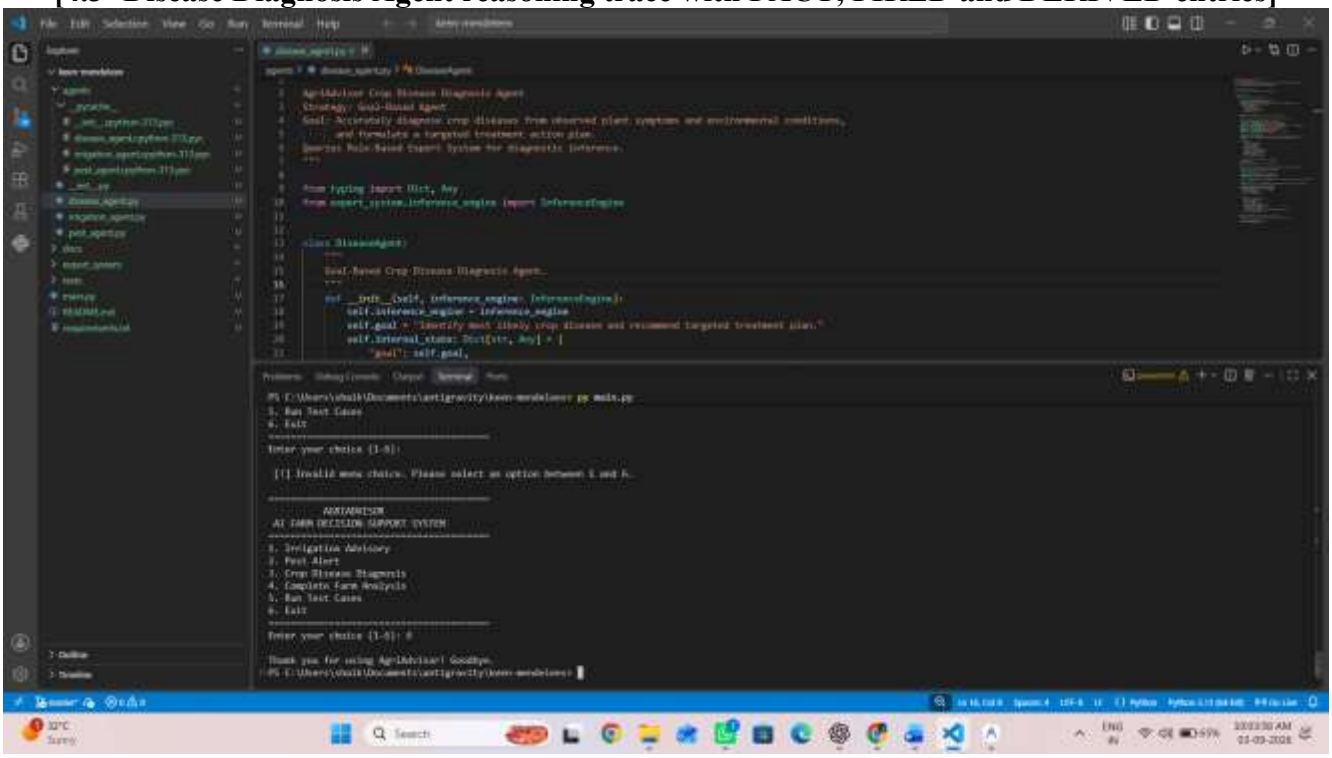
**[Figure 4.1- AgriAdvisor CLI main menu and Complete Farm Analysis output]**



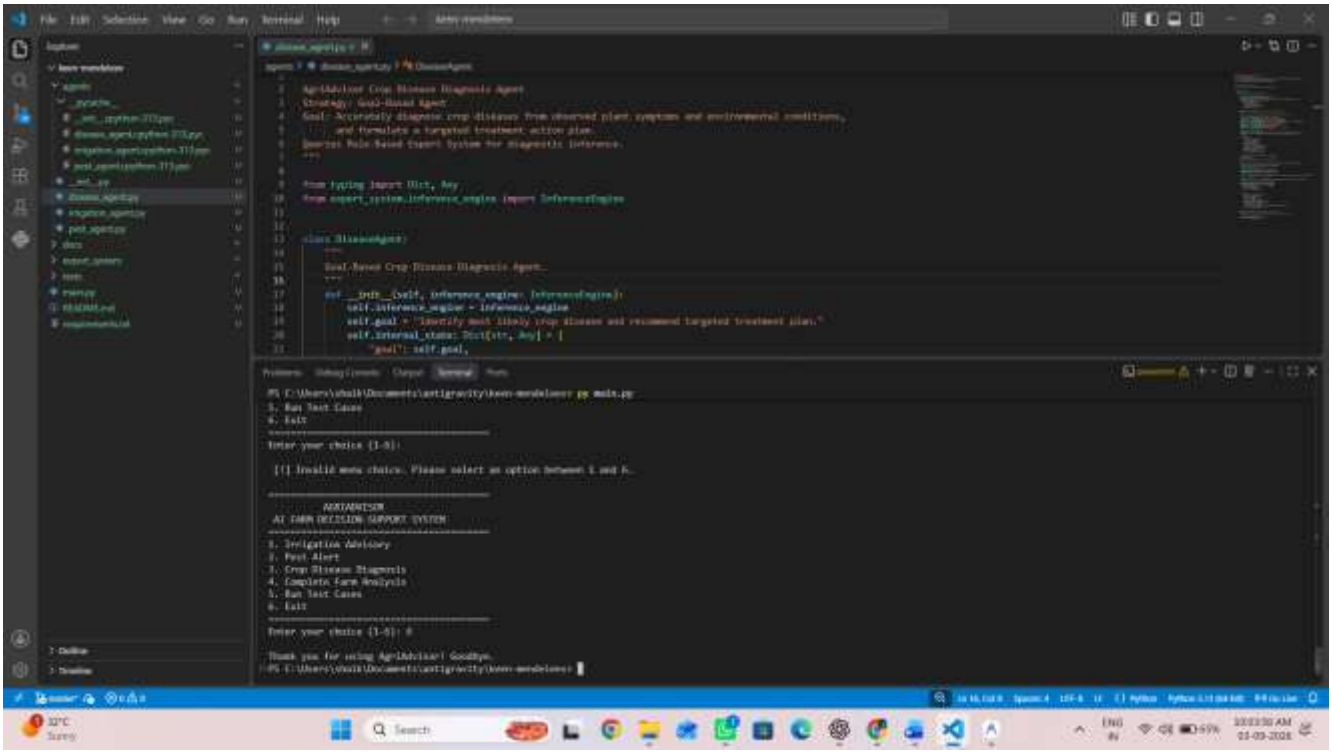
**[Figure 4.2- Automated test suite showing TC01–TC10 PASS and 100.0% pass accuracy]**



**[4.3- Disease Diagnosis Agent reasoning trace with FACT, FIRED and DERIVED entries]**



**[INSERT ACTUAL SCREENSHOT — Figure 4.4]**
**Irrigation Advisory Agent output for low moisture / rainfall scenario**



**[Figure 4.5- Pest Alert Agent output for high-temperature, high-humidity and high-pest-count scenario]**

# CHAPTER 5

# CONCLUSION, FUTURE WORK AND REFLECTION

## 5.1 Conclusion

AgriAdvisor presents a modular multi-agent approach to farm decision support using a rule-based expert system. The system divides the problem into irrigation advisory, pest alerting, and crop disease diagnosis, allowing each sub-task to use an agent strategy suited to its decision characteristics. The Irrigation Advisory Agent uses utility-based reasoning, the Pest Alert Agent uses a model-based reflex strategy with internal state, and the Disease Diagnosis Agent uses a goal-based strategy.

The central expert system contains 40 IF-THEN production rules and a forward-chaining inference engine with specificity-based conflict resolution, numeric priorities, and duplicate/cycle prevention. The agents query this shared reasoning layer and act on its derived recommendations. The automated test suite achieved 10/10 passing cases, demonstrating functional correctness for the implemented scenarios. The result is an explainable academic prototype rather than a certified agricultural advisory product.

## 5.2 Future Work

- Connect the agents to calibrated soil-moisture, temperature, humidity, and rainfall sensors.
- Add validated image-based crop-disease recognition as an additional percept source.
- Integrate local weather forecasts to improve rainfall-aware irrigation utility.
- Expand the knowledge base with more crops, diseases, pests, and region-specific agronomic rules.
- Introduce confidence scoring and expert review for uncertain diagnoses.
- Add multilingual farmer-facing interfaces and optional mobile/web dashboards.
- Evaluate the system through larger field trials with agricultural experts and measured outcomes.

## 5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- PEAS-based problem formulation and agent-strategy selection.
- Design of utility-based, model-based reflex, and goal-based software agents.
- Knowledge-base construction using IF-THEN production rules.
- Forward-chaining inference and conflict resolution.
- End-to-end multi-agent integration and scenario-based testing.

Engineering & Professional Skills Developed:

- System decomposition and modular software design.
- Rule debugging and reasoning-trace analysis.
- Structured testing and evidence-based engineering judgement.
- Technical documentation, teamwork, and presentation preparation.

**Individual Reflection (Shaik Mubarak-192425194)**

Module 1: Irrigation Advisory Agent

**Most difficult engineering problem encountered and how it was solved:**

The most difficult part of my work was converting irrigation decisions into a clear agent strategy rather than using a single threshold. I had to consider soil moisture, rainfall expectation, temperature, crop stage, and the need to avoid unnecessary water use. I solved this by designing a utility-based approach and then connecting the agent to explicit expert-system rules so that each recommendation could be explained.

**A key engineering decision made personally and the evidence behind it:**

My key engineering decision was to keep the irrigation logic inside a separate software agent before passing the relevant facts to the shared expert system. This made the module easier to test independently and allowed the irrigation strategy to maintain its own internal state and utility score.

**New knowledge acquired independently:**

Through this module, I learned how PEAS can be used to analyse an AI problem, how utility-based agents handle competing objectives, and how a rule engine can provide an explainable decision layer. I also learned how test scenarios can be used to verify whether the agent behaves correctly under changing environmental conditions.

**What would be redesigned if the project were repeated:**

If I repeated the project, I would connect the irrigation agent to real soil-moisture and weather sensors, add historical trend data, and validate the utility weights using field observations. I would also add more crop-specific water requirements and expert-reviewed rules.

**REFERENCES**

[1] Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, Pearson, relevant edition.

[2] George F. Luger, Artificial Intelligence: Structures and Strategies for Complex Problem Solving, relevant edition.

[3] FAO, Food and Agriculture Organization of the United Nations, resources on sustainable irrigation, crop production and integrated pest management.

[4] Python Software Foundation, Python 3 Documentation.

[5] Relevant agricultural extension material for crop disease identification and irrigation management used/consulted by the project team.

[6] Relevant research papers on agricultural decision-support systems and rule-based expert systems reviewed by the project team.

[7] Project source code and test outputs maintained in the approved AgriAdvisor GitHub repository.

Note: Before final submission, replace any generic reference description above with the exact title, authors, publisher/URL, and access date of the sources actually consulted by the project team, in accordance with institutional requirements.

## APPENDICES

### Appendix A – Detailed Agent Strategy Summary

| Agent | Strategy | Internal State | Primary Goal / Decision |
|---|---|---|---|
| Irrigation Advisory Agent | Utility-Based | Latest percepts, previous moisture, crop requirement, utility score, irrigation status | Choose the best irrigation-related action |
| Pest Alert Agent | Model-Based Reflex | Previous risk, pest count, trend, environmental state, previous alerts | Update pest risk and issue alert/action |
| Disease Diagnosis Agent | Goal-Based | Symptoms, candidate diseases, evidence, diagnostic state | Identify likely disease and recommendation |

### Appendix B – Agent Percepts and Actions

| Agent | Percepts | Actions |
|---|---|---|
| Irrigation | soil_moisture, temperature, humidity, rainfall, crop, growth_stage, rainfall_expected | START_IRRIGATION, STOP_IRRIGATION, DELAY_IRRIGATION, SEND_IRRIGATION_ALERT |
| Pest | temperature, humidity, rainfall, crop, crop_stage, leaf_damage, pest_count | LOW_RISK, MEDIUM_RISK, HIGH_RISK, SEND_PEST_ALERT, RECOMMEND_CONTROL |
| Disease | crop, leaf_color, leaf_spots, wilting, lesions, leaf_curling, temperature, humidity | DIAGNOSE_DISEASE, RECOMMEND_TREATMENT, MONITOR, SEND_ALERT |

### Appendix C – Rule Catalog (Representative Rules)

R1: IF crop = tomato AND leaf_spots = brown AND humidity = high THEN disease = early_blight

R2: IF crop = tomato AND leaf_curling = yes AND pest_indicator = whitefly THEN disease = tomato_leaf_curl

R3: IF crop = rice AND lesions = diamond THEN disease = rice_blast

R4: IF crop = rice AND leaf_spots = brown THEN disease = rice_brown_spot

R5: IF crop = chilli AND leaf_curling = yes THEN disease = chilli_leaf_curl

R6: IF leaf_surface = white_powder THEN disease = powdery_mildew

R7: IF leaf_spots = water_soaked AND bacterial_signs = yes THEN disease = bacterial_leaf_spot

R8: IF disease = early_blight THEN action = remove_affected_leaves

R9: IF disease = early_blight AND humidity = high THEN action = improve_airflow

R10: IF soil_moisture = low AND rainfall_expected = no THEN irrigation = required

R11: IF soil_moisture = adequate THEN irrigation = not_required

R12: IF soil_moisture = low AND rainfall_expected = yes THEN irrigation = delay

R13: IF temperature = high AND humidity = high AND pest_count = high THEN pest_risk = high

R14: IF pest_count = low AND leaf_damage = none THEN pest_risk = low

R15: IF humidity = high AND temperature = suitable_for_whitefly THEN whitefly_risk = high

R16: IF humidity = high AND temperature = suitable_for_thrips THEN thrips_risk = high

R17: IF pest_count = high THEN action = monitor_pests

R18: IF disease = unknown AND symptoms_incomplete = yes THEN action = collect_more_observations

R19: IF crop = tomato AND leaf_spots = dark AND humidity = high THEN disease = late_blight

R20: IF crop = tomato AND leaf_color = green AND symptoms = none THEN crop_status = healthy

The implemented knowledge base contains 40 rules. The above list provides representative rules; the complete rule definitions are maintained in expert_system/knowledge_base.py and documented in Knowledge_Base.md.

**Appendix D – Software Architecture / Repository Information**

Run the project from the AgriAdvisor root directory:

py main.py

Run automated tests:

py tests/test_cases.py

Repository:
[INSERT APPROVED GITHUB REPOSITORY URL]

**Appendix E – Experimental / Raw Test Data**

| Test | Input Summary | Expected Output | Actual Output | Status |
|---|---|---|---|---|
| TC01 | Tomato + brown spots + high humidity | Early Blight | PASS — as reported | PASS |
| TC02 | Tomato + leaf curling + pest indicators | Leaf Curl | PASS — as reported | PASS |
| TC03 | Rice + blast symptoms | Rice Blast | PASS — as reported | PASS |
| TC04 | Rice + brown spot symptoms | Rice Brown Spot | PASS — as reported | PASS |
| TC05 | Chilli + leaf curling | Chilli Leaf Curl | PASS — as reported | PASS |
| TC06 | Low moisture + no rain | Irrigation Required | PASS — as reported | PASS |
| TC07 | Adequate moisture | No Immediate Irrigation | PASS — as reported | PASS |
| TC08 | Low moisture + rain expected | Delay Irrigation | PASS — as reported | PASS |
| TC09 | High temperature + humidity + high pest count | High Pest Risk | PASS — as reported | PASS |
| TC10 | Healthy crop conditions | Continue Monitoring | PASS — as reported | PASS |

**Appendix F – Ethics / Institutional Approval**

This project is a software decision-support prototype and does not involve human-subject experimentation within the stated scope. Any future field trial involving farmers, farm records, or

identifiable personal information should follow applicable institutional, privacy, and ethical requirements.

**Appendix G – Project Meeting / Progress Records**

| Date | Meeting / Activity | Participants | Decision / Work Completed | Next Action |
|---|---|---|---|---|
| [Date] | Problem definition | [Names] | Finalised AgriAdvisor problem, objectives and sub-tasks | [Action] |
| [Date] | Agent strategy review | [Names] | Selected Utility-Based, Model-Based Reflex and Goal-Based strategies | [Action] |
| [Date] | Rule-engine review | [Names] | Reviewed knowledge base and inference flow | [Action] |
| [Date] | Testing review | [Names] | Reviewed 10 test cases and outputs | [Action] |
| [Date] | Final review | [Names] | Reviewed report and demonstration readiness | [Action] |

**Appendix H – User / Industry Feedback**

No external user/industry feedback was collected within the project scope.

**Appendix I – Publications / Patents / Competitions / Product Outcomes**

No publication, patent, competition or product outcome was claimed for this project.

**Appendix J – Individual Contribution Evidence**

Include evidence supporting individual contributions, such as screenshots of module development, test logs, documentation sections, repository commit history, meeting records, and demonstration outputs.

**CAPSTONE PROJECT OUTCOME MAPPING SHEET**

| Outcome Evidence | CO / Area | Location in This Report |
|---|---|---|
| Problem formulation and PEAS | CO3 | Ch. 1 and Ch. 3 |
| Agent strategy selection | CO3 | Ch. 3.1–3.2 |
| Software agent design | CO4 | Ch. 3.3–3.6 and Appendix A/B |
| Knowledge-base construction | CO5 | Ch. 3.7 and Appendix C |
| Inference engine | CO5 | Ch. 3.6–3.8 and Ch. 4.2 |
| Agent–expert-system integration | CO3/CO4/CO5 | Ch. 4.2 and 4.6 |
| Testing and analysis | CO5 | Ch. 4.4–4.7 and Appendix E |
| Teamwork and project management | CO4 | Contribution Statement and Ch. 4.10 |
| Sustainability and responsible resource use | SDG 2 / SDG 12 | Ch. 3.13 |
| Innovation and infrastructure | SDG 9 | Ch. 1, Ch. 3 and Ch. 5 |